Electronic Commerce Research (2021) 21:371–391
https://doi.org/10.1007/s10660-019-09362-7

Cash flow prediction: MLP and LSTM compared to ARIMA and Prophet

Hans Weytjens[1] · **Enrico Lohmann**[2] · **Martin Kleinsteuber**[2,3]

Published online: 19 July 2019
© Springer Science+Business Media, LLC, part of Springer Nature 2019

Abstract

Cash flow prediction is important. It can help increase returns and improve the allocation of capital in healthy, mature firms as well as prevent fast-growing firms, or firms in distress, from running out of cash. In this paper, we predict accounts receivable cash flows employing methods applicable to companies with many customers and many transactions such as e-commerce companies, retailers, airlines and public transportation firms with sales in multiple regions and countries. We first discuss "classic" forecasting techniques such as ARIMA and Facebook's™ Prophet before moving on to neural networks with multi-layered perceptrons and, finally, long short-term memory networks, that are particularly useful for time series forecasting but were until now not used for cash flows. Our evaluation demonstrates this range of methods to be of increasing sophistication, flexibility and accuracy. We also introduce a new performance measure, interest opportunity cost, that incorporates interest rates and the cost of capital to optimize the models in a financially meaningful, money-saving, way.

Keywords Cash flow prediction · Accounts receivable · Neural networks · LSTM · MLP · ARIMA · Prophet

✉ Hans Weytjens
hans@weytjens.net

Enrico Lohmann
enrico.lohmann@mercateo.com

Martin Kleinsteuber
martin.kleinsteuber@mercateo.com

[1] Gruenwald, Germany

[2] Mercateo AG, Munich, Germany

[3] Technische Universität München (TUM), Munich, Germany

1 Introduction

1.1 Importance and challenges of cash flow prediction

The old adage "cash is king" has not lost any of its relevance: a firm running out of cash will become insolvent. Meanwhile healthy, mature firms also have much to gain from accurate cash flow forecasting: they can maximize their returns by reducing the need for capital, be it in the form of loans, bonds or equity issued, and by allocating the available funds to the most promising projects.

In this paper, we explore the prediction of accounts receivable cash flows from customers. These cash flows have stochastic properties and are not fully under management's control, requiring forecasting to predict their behavior. The data we used contains over 700,000 invoices per year, none of which related to periodic payments. Therefore, our conclusions are valid for companies with a vast amount of non-periodic invoices, such as e-commerce firms, retailers, airlines, logistics companies, etc. The complexity of forecasting the cash flows that emanate from such businesses resides in the large volume of transactions, seemingly random timing of payments, weekends and banking holidays with often regional differences, currency exchange rate variations, as well as transactions' irregularity, as manifested in varying numbers of invoices, sometimes multiple payments, occasional credit notes, prepayments, etc. Moreover, predictions are made to achieve certain goals, which vary depending on the use case. We believe finance managers can, and should, have a tailor-made cash flow prediction system that depends on their specific needs based on factors including the cost of capital, debit and credit interest rates, etc. in order to save money or meet other business objectives. Other considerationse e.g. the forecasting window, the frequency of the forecasts, availability of data, should also be taken into account.

The cash flow prediction system should be performant and flexible enough to overcome this data complexity on one hand and to accommodate these user demands by allowing for customer-specific performance measures and associated optimization on the other.

1.2 Literature

Cash flow forecasting is part of liquidity management and is of the utmost importance for firms [1–4]. To the best of our knowledge, there is no publicly available research on advanced accounts receivables cash flow forecasting, let alone on methods to tune the forecasts to the specific needs as described above. For interested readers, we provide some references to research about related problems and the methods we used below.

Cash flow prediction for projects, mostly construction projects simultaneously considering accounts payable and receivable [5], is a well-researched topic. The stochastic properties of these problems are incomparable to those of the problem of forecasting accounts receivable cash flows of large on-going businesses with

a high volume of invoices stemming from many customers. As a result, the statistical methods employed are not useful for our case. Some cash flow research, often using deep learning methods, also concentrates on predicting whether or not customers will default payments [6, 7], which is a classification problem that does not yield a prediction for the cash flow itself. There is extensive literature about the methods we use in this paper: ARIMA [8, 9], Prophet [10], MLPs [11, 12], LSTMs [11, 13–15]. Papers about demand and sales forecasting, using ARIMA [16], Prophet [17] or LSTM [18, 19] are insightful since these methods of forecasting are highly similar to our cash flow forecasting. Recurrent networks such as LSTMs, however, struggle with very long and event-based sequences [20], preventing us from leveraging the information contained in the individual transactions. Neural networks are subject to ongoing research. We benefited from the recently introduced adam [21] method for stochastic optimization that was deployed in our models. The difficulties associated with analyzing the importance of the individual variables are thoroughly explained in [22], whilst [23] describes how the lack of understanding of deep neural networks' inner workings led them to be considered black boxes. Nevertheless, publications such as [24] offer valuable insights in feature importance. The IOC measure we designed is based on solid financial concepts that have been described in many textbooks [1, 25] and converted into a differentiable loss function, making use of pseudo-Huber loss functions [26].

This paper complements this journal's articles depicting how e-commerce companies can deploy their own data [27] as well as data found in their environment [28] for improved decision making, of which some also use time series methods [29].

1.3 Our contribution

The contribution of this paper is twofold: First, we present and compare a series of existing methods for cash flow forecasting that are not only increasingly sophisticated, but also increasingly accurate and flexible to use. This comparison will enable finance managers to choose the appropriate method for their firms. After a quick glance at Weighted Moving Averaging techniques, we discuss and evaluate two more advanced classical forecasting methods (ARIMA and Prophet). We then propose and evaluate two neural network models (MLP and LSTM) that prove to be very accurate, handle the complexity well and offer flexibility in terms of optimization and data input.

Second, we introduce a new performance measure, Interest Opportunity Cost (IOC). IOC is a measure based on sound financial concepts that allows finance-specific comparisons of forecasting models. Exploiting the aforementioned neural networks' flexibility, we also optimize our MLP and LSTM models using IOC as a cost function, thereby tuning them to produce money-saving predictions. Consequently, from a business perspective, IOC proves to deliver superior results compared to the more classic measures, such as MSE (Mean Squared Error), which are used in most methods.

2 Selection and overview of methods

Our selection out of the wide universe of methods was guided by our objective to offer the finance manager a spectrum of choices of increasing sophistication, accuracy and flexibility. Both linear and non-linear, as well as uni- and multivariate methods were considered. Furthermore, we wanted our methods to be widely understood and deployed, believing this could facilitate their eventual adoption in the finance manager's firm. We start with the simplest of all methods, the Weighted Moving Average, which linearly relates the current value of the time series to its earlier values. Moving up the univariate methods' scale, ARIMA allows the incorporation of seasonality at the expense of higher user-complexity. To take into account holidays with their significant impact on cash flows, we need multivariate methods. Here, Prophet enters the picture as a tool that promises to make forecasting easy and able to deal with holidays, thus offering two improvements on ARIMA. To test whether adding further input variables can further improve our results, neural networks offer virtually unlimited possibilities. We will also use their option for custom-made cost functions. The plain-vanilla neural networks called multi-layer perceptrons (MLP) are our first choice. Long Short-Term Memory (LSTM) networks are specialized in sequences such as time series and will therefore complete our set of methods.

In the following, a model is defined as an equation or set of equations representing reality, expressing the variable to be forecasted as a function of past observations of that particular variable, and possibly others, and of model parameters. These parameters are to be estimated in order to use the model for predictions. The cost function yields a calculated value based on the difference between true values and those predicted by the model. Minimizing the cost function is the criterion used as a goal for estimating the model parameters, called optimization. The cost function for our type of prediction models usually is MSE—the mean of the squared values of the differences between all observed and corresponding predicted values—but we will provide an alternative in Sect. 3. A method combines a model, cost function and optimization technique. Below follows a short description of the methods. More details can be found in appendix.

2.1 Weighted moving average

The Weighted Moving Average method's simplicity is its most attractive feature and explains its popularity. The target variable is expressed as a linear combination of earlier instances of the same variable. The model is thus of type $\hat{y}_t = f(\boldsymbol{y}_{t-1})$, with $\hat{y}_t$ the forecasted cash flow at time t, $\boldsymbol{y}_{t-1}$ a vector of cash flow observations $[y_{t-1}, \ldots, y_{t-n}]$ and f the model function.

2.2 ARIMA

The Auto-Regressive Integrated Moving Average (ARIMA) [8, 9] method uses a higher level of univariate models of the type $\hat{y}_t = f(\boldsymbol{y}_{t-1})$. In contrast to the Weighted Moving Average method, the general forecasting equations are no longer linear,

and iterative techniques are to be used to find solutions. Upon closer inspection, the basic version of ARIMA equals the Weighted Moving Average method. For this reason, the latter will no longer be considered in this paper. ARIMA is robust, able to cope with trends and seasonality but requires some parameter tuning. Many publicly available software implementations facilitate its implementation. ARIMA is, therefore, widely used for time series.

2.3 Prophet

Facebook™ made its Prophet forecasting tool available to the public in early 2017. In the article announcing Prophet citeprophet, the authors justify the decision after observing that "completely automatic forecasting techniques can be brittle, and they are often too inflexible to incorporate useful assumptions or heuristics" and that "analysts who can produce high quality forecasts are quite rare because forecasting is a specialized data science skill requiring substantial experience." In addition to the ease of use, Prophet promises to handle trends and multiple seasonalities and to deal better with outliers, missing observations and trend changes. Prophet is based on a model of type $\hat{y}_t = f(\boldsymbol{y}_{t-1}, \boldsymbol{hol})$, with $\boldsymbol{hol}$ a user-provided set or vector of holidays for the firm.

2.4 Multi-layer perceptrons

The current success and resulting popularity of deep learning can be attributed to the preceding and ongoing explosion of data availability, storage capacity and raw computing power. Artificial neural networks were originally inspired by an increasingly detailed understanding of the human brain. The digital equivalent of the neuron is a node. Nodes are connected by links.

Information in feedforward networks or perceptrons only flows in one direction—from input to output. The nodes are organized in layers. Every node produces an output applying a non-linear function to its inputs from the layer below. A multi-layer, or deep feed forward network, is also called a multi-layer perceptron (MLP). It consists of one or more hidden layers between the input and output layers. An example can be seen in Fig. 1.

MLPs, as all neural networks, are used to estimate the true, unknown function that explains the output vector $\boldsymbol{y}$ in function of the input vectors $\boldsymbol{x}$ [11]. This involves optimizing the parameters $\boldsymbol{\Theta}$, so that the function $f(\boldsymbol{x};\boldsymbol{\Theta})$ approximates this true function. After an input is fed to a neural network, information will start flowing through it, producing values for the nodes in the output layer. These values are then used to compute the scalar cost function $J(\boldsymbol{\Theta})$. The cost function is often MSE, but neural networks allow for custom-made cost functions as will be discussed in Sect. 3. Training the neural network implies finding the parameters $\boldsymbol{\Theta}$ that minimize the cost function $J(\boldsymbol{\Theta})$.

Compared to the classical models above, MLPs are models of the more generic type $\hat{\boldsymbol{y}} = f(\boldsymbol{x})$ in which the vector $\boldsymbol{x}$ can include all possible quantifiable variables, amongst which, but not limited to: $\boldsymbol{y}_{t-1}, \boldsymbol{hol}$, etc.

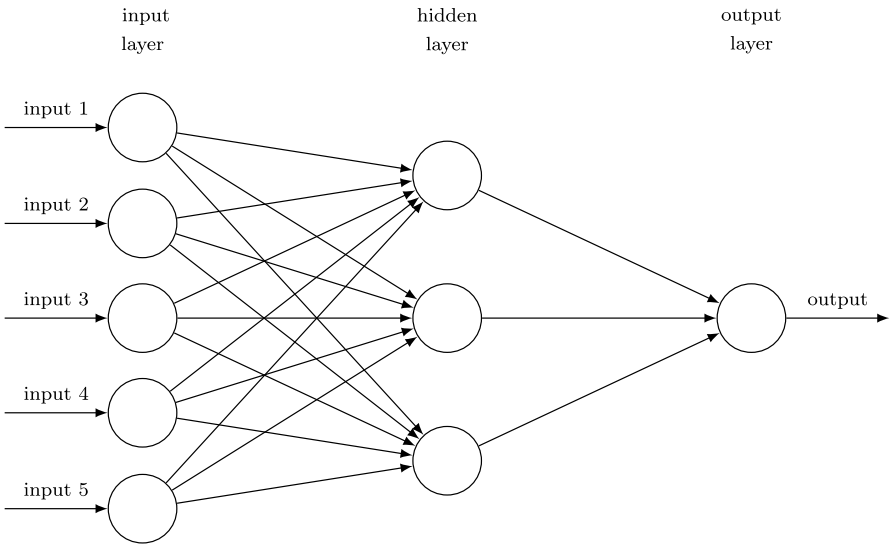


Fig. 1 Example of a multi-layer perceptron (MLP) with five input nodes, one hidden laye with three nodes and one output node. Inputs are forward-propagated following the arrows

2.5 Recurrent neural networks with a focus on LSTM

An MLP considers only one event at a time and assumes all inputs to be temporarily independent of each other. This assumption, however, is invalid in many real-world situations: In speech, every word in a sentence is related to one another, while data in time series, such as cash flows, is also related to past data. Recurrent neural networks (RNNs) are designed to handle these dependencies; hence, they are widely used in speech recognition, translation, image captioning, video-to-text, question answering, time series forecasting, etc.

At any time step, an RNN node will remember information from earlier time steps as visualized in Fig. 2. RNNs allow for sequences in the output, input, or both, whereas MLPs are only capable of handling fixed-length vectors as input and output. As an improvement on the basic RNN models, Long Short-Term Memory models (LSTMs) [13] are currently the most used form of RNN models.

LSTMs, thus, belong to the $\hat{\boldsymbol{Y}} = f(\boldsymbol{X})$ class of models, in which $\boldsymbol{Y}$ and $\boldsymbol{X}$ are matrices. Given their complexity, however, they will take longer to train than equivalent MLPs and will require substantially more computer resources.

The result of all deep learning models, including MLPs and LSTMs, will be of a stochastic nature, as the initial parameters $\boldsymbol{\Theta}$ are usually random and the learning process itself also involves random elements. Therefore, it is recommended to always train several models, make predictions from them and then take an average of the results.

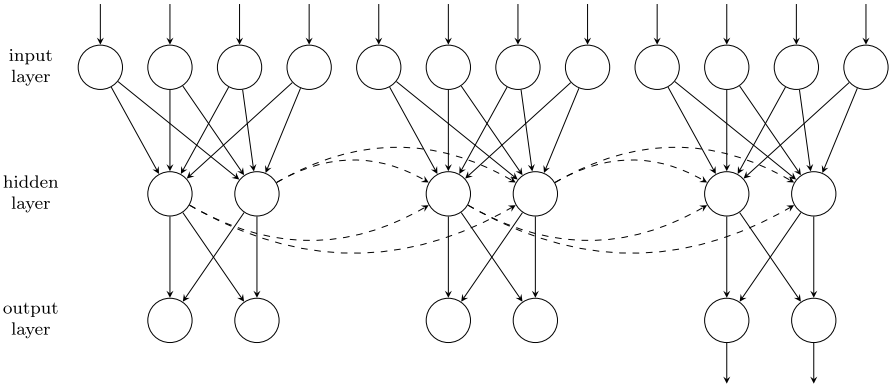


Fig. 2 Example of a recurrent neural network (RNN) with four input variables in three time steps, one hidden layer with two nodes and two output variables in one time step. The nodes in the hidden layer pass information through subsequent instances as shown by the dotted arrows

3 A new performance measure: interest opportunity cost

As an alternative to MSE, we propose a new, more targeted measure: the Interest Opportunity Cost (IOC). A firm's finance manager will want to minimize the money residing in the customer payments' bank account to be able to transfer it to an interest-yielding savings account or, alternatively, minimize the firm's working capital, thus minimizing an opportunity cost i_{debit}. At the same time the finance manager will want to avoid overdrafting the account, which costs i_{credit}, leading to an opportunity cost of $(i_{\text{credit}} - i_{\text{debit}})$. IOC is consequently defined as:

$$\text{IOC} = \frac{1}{n}\sum_{\substack{j=1 \\ \varepsilon_j>0}}^{n} \varepsilon_j i_{\text{debit}} - \frac{1}{n}\sum_{\substack{j=1 \\ \varepsilon_j<0}}^{n} \varepsilon_j \left(i_{\text{credit}} - i_{\text{debit}}\right) \tag{1}$$

with n, the number of predicted values, and $\varepsilon_j = y_j - \hat{y}_j$ the prediction errors. The measure's interpretation follows from this definition. IOC expresses in monetary terms (e.g. EUR if the underlying data are in EUR) how much money is lost using a model's imperfect predictions compared to the hypothetical case of perfect predictions. In an ideal scenario with perfect forecasting, IOC would be zero. Figure 3[1] demonstrates the differences between IOC and both MSE (mean squared error) and MAE (mean absolute error). The parameters for IOC ($i_{\text{debit}} = 8\%$ and $i_{\text{credit}} = 10\%$) will also be used in our simulation in Sect. 5. With these parameters, IOC is clearly asymmetrical, leading us to anticipate that our models will show a preference towards $\varepsilon < 0$, corresponding to overestimates, which suggest a preference for overdrafting rather than keeping too much available cash.

[1] For scaling purposes in this figure, the IOC values were multiplied by 20.

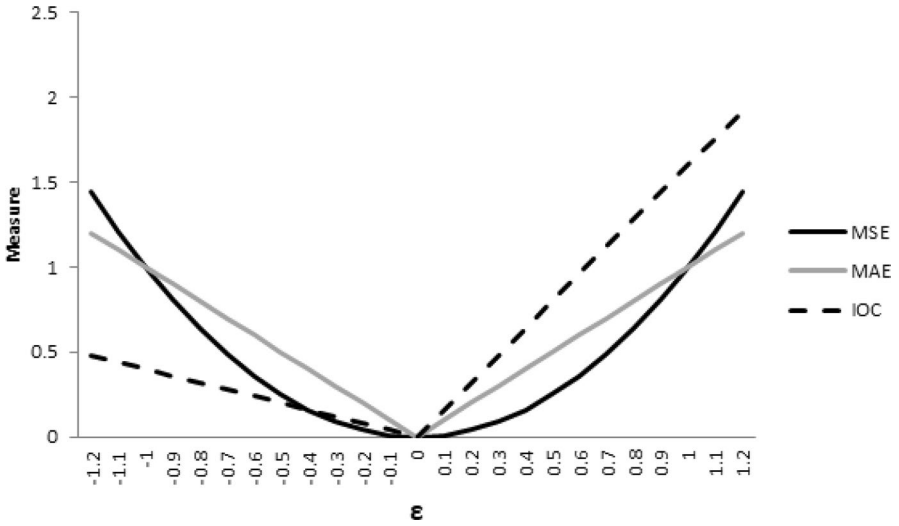


Fig. 3 Performance measures: IOC compared to MSE and MAE. Note the different forms of asymmetry for IOC

Neural networks are extremely powerful methods and changing their cost function could have a noticeable effect. At the same time, they are quite flexible and most software implementations allow customized cost functions. We, therefore, optimized both our MLP and LSTM models by minimizing IOC next to the more standard MSE. This was implemented using a customized weighted compounded pseudo-Huber cost function as an approximate function to make IOC differentiable:

$$loss = \sum_{j=1}^{n} \left(w_{\mathrm{j},+} h_{\mathrm{j}}(i_{\mathrm{debit}}) - w_{\mathrm{j},-} h_{\mathrm{j}}\left(i_{\mathrm{credit}} - i_{\mathrm{debit}}\right)\right), \text{ in which:}$$

$$w_{\mathrm{j},\pm} = 0.5 + \frac{\tanh\left(\pm \varepsilon_{\mathrm{j}} a^2\right)}{2}, h_{\mathrm{j}}(x) = x^2 \left(\sqrt{1 + \left(\frac{\varepsilon_{\mathrm{j}} a}{x}\right)^2} - 1 \right), \text{ with } a = 1000. \tag{2}$$

IOC, thus, enables optimizing the models according to a financially meaningful performance measure. Since i_{debit} and i_{credit} are parameters to the model, they can be modified as often as desired to reflect changing interest rates in the market.

4 The data

Our data consists of transactions within Germany for the period June 1st, 2015—February 28th, 2018. It was anonymized and does not include bank account data. Table 1 presents the transaction dataset's features we used. The first three fields are self-explanatory. INV_AMNT_EUR is interpretated in function of the

Table 1 Transactions dataset: selection of features

Feature	Used	Explanation	Type
UNIQUE_ID	–	Uniquely identifies transaction	String(11)
TRANS_DATE	✓	Date of event/item	Date
CUST_ID	✓	Uniquely identifies customer	String(10)
TRANS_TYPE	✓	Invoice (INV), payment (PAY), prepayment (PRE), credit note (CRN), ...	String(3)
INV_AMNT_EUR	✓	Item amount in euro	Float
CAL_DAYS_DUE	–	Number of days payment is due after invoice date	Integer
CURR	–	Currency	String(3)

CUST_ID connects to the customer dataset. The value of TRANS_TYPE determines whether the value of INV_AMNT_EUR qualifies as a payment

Table 2 Example of a complete transaction, which consists of two invoices on June 1st and 4th followed by their payment on July 21st (selected columns)

UNIQUE_ID	TRANS_TYPE	TRANS_DATE	INV_AMNT_EUR
3IH3QDZK4O0	INV	2015-06-01	– 0.000540
3IH3QDZK4O0	INV	2015-06-04	– 0.000104
3IH3QDZK4O0	PAY	2015-07-21	0.000644

Table 3 Customer dataset: selection of features

Feature	Used	Explanation	Type
CUST_ID	✓	Uniquely identifies customer	String(10)
POSTAL_CODE	✓	Zip code	Integer
CITY	–	City	String
COUNTRY	–	Country	String(2)

POSTAL_CODE points to the customer's state (Bundesland)

corresponding value of TRANS_TYPE, e.g. when the value of TRANS_TYPE is "PAY", then INV_AMNT_EUR will be the amount paid by the customer. The data is grouped by transactions. Every transaction contains at least two items: usually, an invoice and a payment. Table 2 shows an example of a transaction (with a late payment). In total, our transactions dataset contains nearly 5m rows for 1.8m transactions.

We also had access to a customer dataset—shown in Table 3—that we eventually used the POSTAL_CODE feature from to infer the state (Bundesland) and then compute the synthetic feature REL_HOL (see below). Since Germany has 16 states, each with their own public holiday schedule, our methods will work for any multinational company, or company dealing with customers in different geographies.

As the data described above possesses different scaling, we normalized it to be between 0 and 1 before aggregating it on a daily basis, resulting in one row per day.

Table 4 Dataset: synthetic features

Feature	Explanation	Type	AR	Pr	ML	LS
CASH_IN	Sums all forms of payment for the day (feature to be predicted)	Float	✓	✓	✓	✓
IS_WEEKENDDAY	Weekend day	Boolean	–	✓	✓	✓
IS_HOLIDAY	Holiday for the firm	Boolean	–	✓	✓	✓
REL_HOL	Share of sales in state with holiday (customer perspective), computed using POSTAL_CODE and external holiday calendars	Float	–	–	✓	✓

CASH_IN is the feature to be predicted and only variable for ARIMA. Prophet is aware of weekends and holidays, therefore does not require IS_WEEKENDDAY and IS_HOLIDAY. The neural network models use all four features listed in this table

This data was then augmented by including synthetic, i.e. computed features and features derived from other sources as additional inputs to the models as shown in Table 4.[2] In this table, AR, Pr, ML, and LS stand for ARIMA, Prophet, MLP and LSTM, respectively. CASH_IN is the sum of all INV_AMNT_EUR of the respective day for which TRANS_TYPE equals "PAY" or "PRE". IS_WEEKENDDAY is 0 unless the considered day is a weekend day from the firm's perspective in which case it becomes 1. IS_HOLIDAY follows the same concept for holidays. REL_HOL looks at holidays from the customers' perspective. It is 0 if the considered day is a working day for all states in which the firm is active, 1 if that day is a national holiday (i.e. holiday in all states), and the share of states on holiday weighted for their total sales for all intermediate cases. A value of 0.6, for example, means that states generating 60% of the firm's sales are on holiday that day.

5 Results

We ran our models[3] with the inputs described in Table 4. The output data was scaled such that 100 was the maximum daily cash inflow in the dataset. Both MLP and LSTM models were optimized using both IOC and MSE as their cost functions. After testing other options, we decided to use 28-day sequences (per sample), 500 epochs, batch sizes of 112 and adam (adaptive learning rate optimization algorithm [21]) to train the neural networks. To neutralize the stochastic variance of their results, 20 iterations were performed, before calculating the average of their predictions made for evaluation. For IOC, we used 8% for the debit interest rate (here cost of capital) and 10% for the credit interest rate (overdraft). The models were used to step-wise forecast one day into the future, for the 100 days as of September 1st, 2017. Based on our analysis in Sect. 3, we primarily used IOC to evaluate the

[2] The Prophet package has inherent knowledge of weekend days and German holidays.

[3] We used the Python SARIMAX implementation (sm.tsa.statespace.SARIMAX) for ARIMA and Prophet's Python implementation (fbprophet.prophet).

Table 5 Results for one-day forecasts in a series of 100 days from September 1st, 2017

Methods		Measures			
Name	Optimizer	IOC	MSE	MAE	Under/over
Advanced statistical methods and packages					
ARIMA	N/A	0.23	74.04	4.20	61/39
Prophet	N/A	0.21	30.41	3.89	52/48
Neural networks					
MLP	MSE	0.24	32.36	3.90	51/49
MLP	IOC	0.10	33.94	3.14	19/81
LSTM	MSE	0.23	32.97	3.58	56/44
LSTM	IOC	0.09	17.94	2.73	18/82

The neural networks were optimized using both IOC and MSE. The IOC-optimized LSTM achieved the best results

models from a business perspective. Nevertheless, we also computed MSE, mean average error (MAE) and the number of under- and overestimates for additional insights. The results are summarized in Table 5.

It can be seen that both LSTM and MLP optimized for IOC, substantially outperform the other methods. When optimizing for IOC is not an option, the difference between the four remaining models becomes statistically insignificant. Neural networks' inherent ability to optimize along custom-made cost functions clearly pays off. IOC optimization yields biased predictions that tend to overestimate more than underestimate, which corresponds to our expectations as formulated in Sect. 3. When looking at MSE to evaluate our models, ARIMA clearly underperforms due to the squaring of the errors for the holidays, that it cannot account for. According to this measure, LSTM is the clear winner.[4]

The ranking of the models, according to their IOC score, does not change when making the forecasting horizon longer, as shown in Fig. 4.[5] Note that the 30-day LSTM forecast is on par with and even outperforms ARIMA's and Prophet's one-day forecasts. Finally, Fig. 5 reveals where the predictions of our best model, LSTM, deviate most from the corresponding true values. The cash inflows around and on the German national holiday of October 3rd, were very accurately forecasted. The model's worst forecast, accounting for most of its IOC and MSE, occurred the day after October 31st. This holiday was an one-off holiday that only happened in 2017 and had a distinctly different cash flow pattern. Given the paucity of national holidays in the data, some effects on and around holidays were obviously not learned.

Analyzing the importance of the individual features is a challenging task [22]: Deep neural networks are known as black boxes, whose inner workings

[4] LSTM achieves the best MSE and MAE values after optimizing for IOC. This counter-intuitive result can be attributed to the intricacies of the interplay between model, data, cost function, optimization algorithms, etc. The discussion of the phenomenon is outside the scope of this paper.

[5] For clarity, the MSE-optimized MLP and LSTM were omitted.

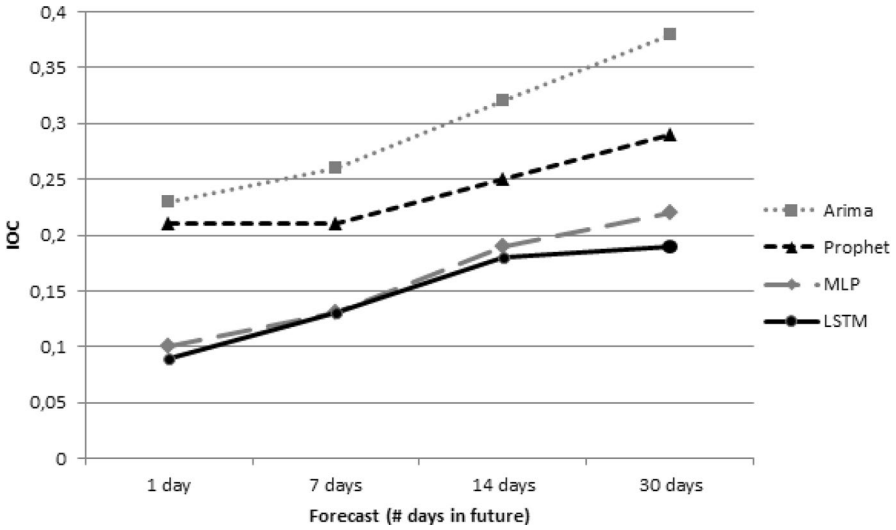


Fig. 4 IOC in function of forecasting horizon length. All methods lose accuracy with longer forecasting horizons, but the order does not change. A 30-day LSTM forecast is still more accurate than a one-day ARIMA or Prophet forecast

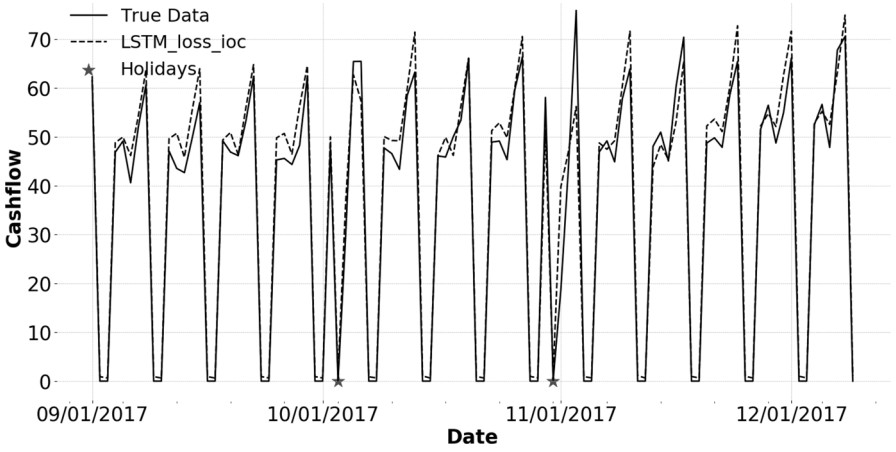


Fig. 5 LSTM with IOC: true versus predicted. The tendency to overestimate is clearly illustrated. The regular normal holiday on October 3rd and the days around it are well forecasted. However, the model struggles with the days after the one-off national holiday on October 31st

are notorioulsy difficult to interpret [23]. As described in [24], a model-agnostic permutation feature importance algorithm can shed some light on the respective importance of the features used. The idea is to measure the impact of shuffling (permuting) a feature's values on the model's performance measure. The feature's own effect and its interaction with the other features in the model are thus taken into account. We ran this algorithm using our IOC-optimized LSTM model for

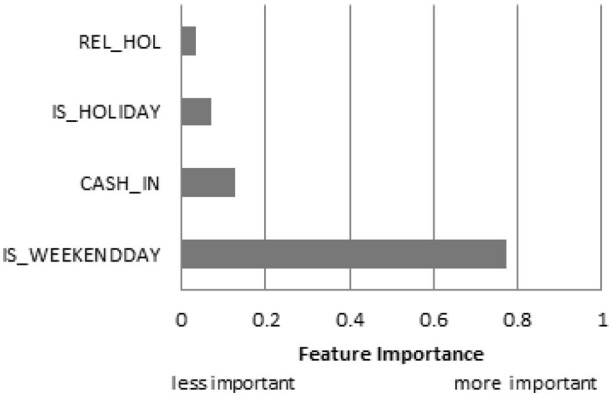


Fig. 6 Features importance comparison for the LSTM model optimized by IOC based on the 100-days test set. For prediction purposes, IS_WEEKENDDAY is most important, all other features are relevant as well

all four features on the 100-days test set, as shown in Fig. 6. IS_WEEKENDDAY turns out to be the most important feature for prediction, whilst CASH_IN comes in a distant second. We explain the high relative importance of IS_WEEKENDDAY by the fact that permuting its values upsets the model most as cash inflows during weekends equal zero. Further analysis revealed that this weekend effect could have been picked up nearly entirely by CASH_IN when dropping IS_WEEKENDDAY from the features list. However, in that scenario IOC would have suffered, hence the feature was retained in our final model.

As neural networks are such powerful tools, we explored several options to further improve our results. We looked at the individual transactions and created models to predict them so that they could then be aggregated company-wide. However, the link between the due and payment dates proved too weak to surpass the level of accuracy achieved by our models working on daily data as described in this paper. As [20] notes, LSTMs find it challenging to cope with very long and event-based sequences, which our transactions obviously are: some easily surpass 100 or more days. Moreover, the transactions' irregularity (multiple invoices, multiple payments, prepayments, etc.) further complicated matters. We also tried to incorporate numerous other variables, but none of them contributed to further increasing the accuracy.

6 Conclusion

We evaluated the above methods' performance from a user's perspective using the following criteria:

- Accuracy: represents the model's capability to achieve goals set out for it as defined by the chosen performance measure

Table 6 Comparative evaluation of cash flow forecasting methods in which ○ stands for low, ◐ for medium and ● for high

Method	Accuracy	Flexibility	Ease	Transparency
Advanced statistical methods & packages				
ARIMA	◐	○	◐	◐
Prophet	◐	○	◐	○
Deep learning methods				
MLP	●	◐	○	◐
LSTM	●	◐	○	◐

- Optimization flexibility: allows for the adaption of the model's intent to align with the business objectives as described in Sect. 3. Flexible models can be optimized in function of different, even user-defined (e.g. IOC), measures
- Ease of use: involves the development, installation, use of hardware resources and maintenance requirements of the model
- Transparency: permits the user to understand how the model arrived at its predictions and generally favors its acceptance.

The first criterion is quantitative and explained above, while the other three are more subjective and are based on our experience using the methods. The results are displayed in Table 6.

Due to the large number of daily transactions in our data, the resulting cash flows have a strong weekly pattern that enabled us to predict them well. The primary challenge arose from holidays: national holidays and holidays specific to each of the different Laender (states in Germany). Since we had less than three years' worth of data available for our research, some residual variances due to holidays could not be fully explained by our models.

In summary, we have presented very useful forecasting models for firms with many customers and many transactions such as e-commerce companies, retailers, airlines and public transportation firms operating in multiple regions. Moving from simple to more sophisticated, we increased the accuracy and, as such, offer finance managers a variety of forecasting models. Moreover, we introduced a new performance measure, IOC, which can be used as a financially meaningful optimizer for the MLP and LSTM models with the aim of saving money. Its parameters, i_{debit} and i_{credit}, can be modified as frequently as desired to reflect changing interest rates and cost of capital in the market.

On behalf of this paper's authors, the corresponding author states that there is no conflict of interest.

Appendix: The methods in more detail

The following notation will be used:

f:	model function
y_t:	cash flow at time t, $\hat{y}_t$ when forecasted
$\boldsymbol{y}_{t-1}$:	vector of cash flow observations $[y_{t-1}, \ldots, y_{t-n}]$
r_i, ϕ_i, θ_i:	model parameters to be estimated
n:	length of time series used in model
p, d, q:	parameters for ARIMA
c, δ:	constants to be estimated
ϵ_t:	error term for prediction of y_t
L:	lag operator, e.g. $L^d y_t = y_{t-d}$
hol:	user provided set or vector of holidays for the firm.

Weighted moving average

The weighted moving average model is of type $\hat{y}_t = f(\boldsymbol{y}_{t-1})$:

$$\hat{y}_t = \sum_{i=1}^{n} r_i y_{t-i}, \text{ with } \sum_{i=1}^{n} r_i = 1 \tag{3}$$

The model parameters r_i can be manually set without use of a cost function, with recent values usually given more weight than older ones. They can also be estimated using a linear regression (MSE). In order to pick up a linear trend, one could adapt the formula to:

$$\hat{y}_t = c + \sum_{i=1}^{n} r_i y_{t-i} \tag{4}$$

Relaxing the constraint $\sum_{i=1}^{n} r_i = 1$ would allow for an exponential trend. Setting all $r_i = 1/n$ would yield a non-weighted moving average. It is obviously not possible to take into account any information other than past observations of the variable to be predicted.

ARIMA

The Auto-Regressive Integrated Moving Average (ARIMA) [8, 9] method uses a higher level of univariate models of the type $\hat{y}_t = f(\boldsymbol{y}_{t-1})$. The model, in a convenient notation ARIMA(p, d, q), is given by:

$$\left(1 - \sum_{i=1}^{p} \phi_i L_i\right)(1 - L)^d y_t = \delta + \left(1 + \sum_{i=1}^{q} \theta_i L_i\right)\epsilon_t \tag{5}$$

in which ϕ_i and θ_i are the parameters of the Auto-Regressive (AR) and Moving Average (MA) parts of the model, respectively.

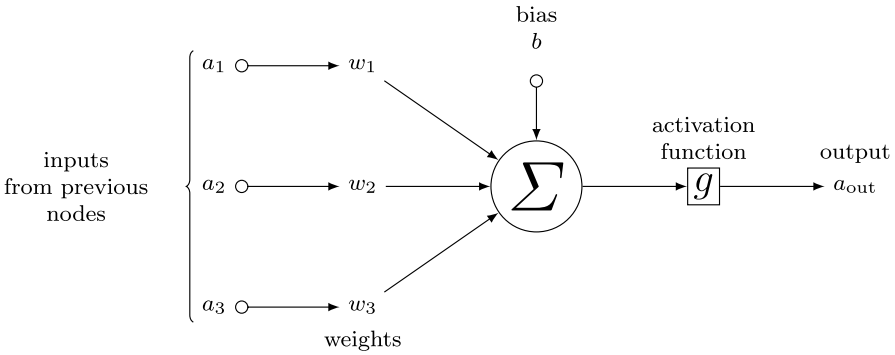


Fig. 7 Calculating the output value a_{out} of a node, as a linear combination of its inputs a_{i} to which a bias b is added before the activation function g is applied

In a first Integration (I) step, the method aims to remove trends and, as such, make the time series stationary. This is done by differencing, a technique whereby the values of consecutive observations are subtracted and possibly complemented by using logarithms to stabilize the variations. E.g. For $d = 2$, a second-degree difference or a difference of differences:

$$(1 - L)^{\text{d}} y_{\text{t}} = y_{\text{t}} - 2y_{\text{t-1}} + y_{\text{t-2}} = (y_{\text{t}} - y_{\text{t-1}}) - (y_{\text{t-1}} - y_{\text{t-2}}) \tag{6}$$

The general equation for the AR and MA parts can be written as:

$$y_{\text{t}} = \delta + \phi_1 y_{\text{t-1}} + \phi_2 y_{\text{t-2}} + \cdots + \phi_{\text{p}} y_{\text{t-p}} + \theta_1 \epsilon_{\text{t-1}} + \theta_2 \epsilon_{\text{t-2}} + \cdots + \theta_{\text{p}} \epsilon_{\text{t-q}} + \epsilon_{\text{t}} \tag{7}$$

This is for the case $d = 0$. For other cases, substitute y_{t} for $(1 - L)^{\text{d}} y_{\text{t}})$. Upon closer inspection, ARIMA $(n, 0, 0)$ equals the Weighted Moving Average method described above. Its parameters can be determined using a linear regression, implicitly assuming an MSE cost function. If, however, the MA part of ARIMA is used and $q > 0$, then the error terms ϵ enter the equation. Since they are the difference between the forecasts and true values, the general forecasting equations are no longer linear, and iterative techniques are to be used to find solutions. To determine the parameters p, d, and q, a grid search can be conducted, and the best performing combination withheld for prediction.

Prophet

Detailing Facebook™ Prophet's exact inner workings would exceed the scope of this paper. However, an additive regression underpins the method, completed by the extraction of a linear or logistic growth trend, Fourier series, the use of dummy variables and a list of holidays. In our use case, Prophet is based on a model of type $\hat{y}_{\text{t}} = f(\boldsymbol{y}_{\text{t-1}}, \boldsymbol{hol})$.

Multi-layer perceptrons

The basic building block of a neural network is the node. Nodes are connected by links, albeit in a more orderly fashion than neurons in the brain. A weight w_i is attributed to each of n links as a measure of its strength and sign. Figure 7 illustrates how the value of a node a_{out} is computed by first calculating the weighted sum of its inputs and then applying a non-linear activation function g to it:

$$a_{out} = g(b + \sum_{i=1}^{n} w_i a_i) \tag{8}$$

To facilitate further reading, we introduce the most important additional notation used:

- $\boldsymbol{y}$: dependent variable: vector of cash flow observations, $\hat{\boldsymbol{y}}$ when forecasted
- $\boldsymbol{x}$: independent variable: vector of cash flow observations and other inputs if any
- $\boldsymbol{\Psi}^j$: represents the function (weighted sum) for each node at layer j
- $\circ$: operator for a composition of functions
- g_j: activation function for layer j
- $\boldsymbol{W}_j$: jth layer's weight matrix containing weights for the layer's nodes (= model parameters to estimated)
- $\boldsymbol{b}_j$: vector of biases for layer j
- J: cost function yielding a scalar
- $\boldsymbol{\Theta}$: parameter space containing all weights and biases for all nodes
- $\boldsymbol{a}_j$: vector of node outputs values from previous layer $j-1$.

Information in feed forward networks or perceptrons only flows in one direction—from input to output. The nodes in these networks are usually organized in layers, without links between the nodes within a layer, such that a node only receives information from (some of) the nodes in the preceding layer. A multi-layer, or deep feed forward network, is also called a multi-layer perceptron (MLP). It consists of one or more hidden layers between the input and output layers.

MLPs, as all neural networks are used to estimate the true, unknown function that explains the output $\boldsymbol{y}$ in function of the inputs $\boldsymbol{x}$ [11]. This involves optimizing the parameters $\boldsymbol{\Theta}$, so that the function $f(\boldsymbol{x};\boldsymbol{\Theta})$ approximates this true function. After an input is fed to a neural network, information will start flowing through it, known as forward propagation, finally producing values for the nodes in the output layer. These values are then used to compute the scalar cost function $J(\boldsymbol{\Theta})$. Mathematically, we can describe MLP as a model:

$$\begin{aligned} &\hat{\boldsymbol{y}} = f(\boldsymbol{x}) = (\boldsymbol{\Psi}^m \circ \boldsymbol{\Psi}^{m-1} \circ \boldsymbol{\Psi}^{m-2} \circ \cdots \circ \boldsymbol{\Psi}^1)(\boldsymbol{x}) \\ &\boldsymbol{\Psi}^j(\boldsymbol{a}_j) = g_j(\boldsymbol{W}_j^T.\boldsymbol{a}_j + \boldsymbol{b}_j), \text{ with } \boldsymbol{a}_1 = \boldsymbol{x} \\ &\text{minimizing the cost function: } \arg\min_{\boldsymbol{\Theta}} J(|\hat{\boldsymbol{y}} - \boldsymbol{y}|, \boldsymbol{\Theta}) \end{aligned} \tag{9}$$

Training the neural network implies finding the parameters Θ that minimize the cost function $J(\Theta)$. This is commonly achieved by a backpropagation algorithm [12] that computes the gradient of $J(\Theta)$ with respect to all weights in the neural network. Another algorithm, for example SGD (stochastic gradient descent) or adam (adaptive moment estimation) [21], will then utilize these gradients to adjust all parameters Θ. Once that is done, a new set of inputs is run through the network, and the entire learning cycle repeats itself until a pre-determined stopping criterion is satisfied.

Compared to the classical models above, MLPs are models of the more generic type $\hat{\boldsymbol{y}} = f(\boldsymbol{x})$ in which the vector $\boldsymbol{x}$ can include all possible quantifiable variables, amongst which, but not limited to: $\boldsymbol{y}_{t-1}, \boldsymbol{hol}$, etc. If we were to build a model to forecast the cash inflow based on the previous day's cash inflow and the holiday status of that day, the model would be: $\hat{y}_t = f(y_{t-1}, hol_{t-1})$. If, however, we wanted to input multi-variate time series into an MLP model (2-dimensional input), we would have had to flatten the input into a one-dimensional vector. The above example, thus, becomes $\hat{y}_t = f(y_{t-1}, hol_{t-1}, \ldots, y_{t-k-1}, hol_{t-k-1})$ for a k-day time series. We used this technique when evaluating MLPs for cash flow forecasting. Our output vector of length 1 corresponds to one node in the last layer.

Recurrent neural networks with a focus on LSTM

An MLP considers only one event at a time and assumes all inputs to be temporarily independent of each other.[6] This assumption, however, is invalid in many real-world situations: In speech, every word in a sentence is related to one another, while data in time series, such as cash flows, is also related to past data. Recurrent neural networks (RNNs) are designed to handle these dependencies; hence, they are widely used in speech recognition, translation, image captioning, video-to-text, question answering, time series forecasting, etc. We propose some additional notation for this section:

- $\boldsymbol{Y}$: matrix with (time) series of dependent variables, $\hat{\boldsymbol{Y}}$ when forecasted
- $\boldsymbol{X}$: matrix with (time) series of independent variables: cash flow observations and other inputs if any
- f_h: function that computes the hidden state of a RNN's node
- h_t: hidden state of a RNN cell at timestep t
- $\boldsymbol{a}_t$: vector of node outputs values from previous layer at timestep t
- σ: sigmoid function.

At any time step, an RNN node will remember past information by having a hidden state h_t that is a function of the hidden state of the previous time step h_{t-1} and the

[6] The flattening technique described in the previous section is only a partial workaround. Since no information is passed between nodes within a layer, the relation beween the different time vectors remains challenging to model in an MLP.

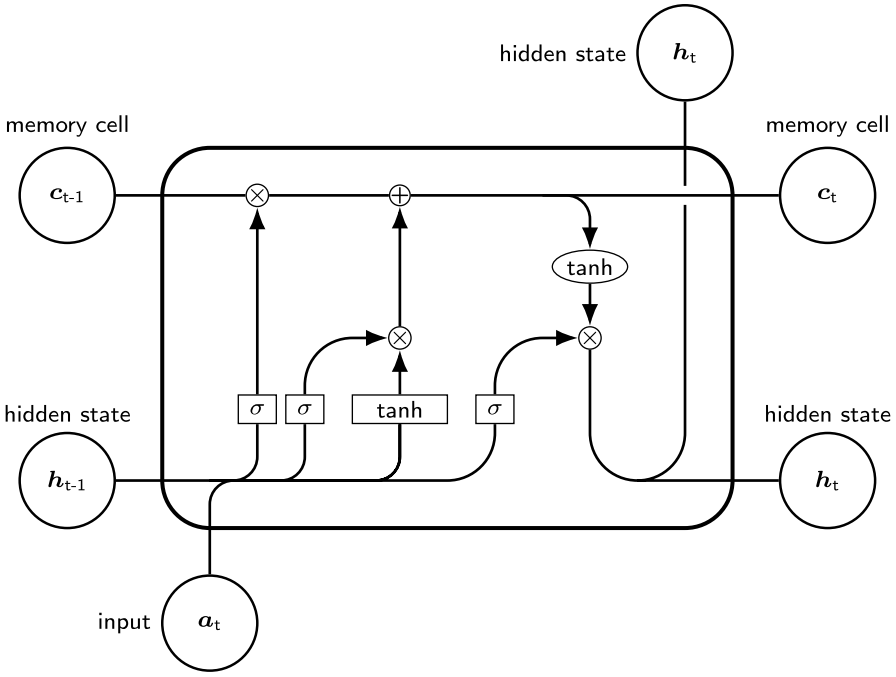


Fig. 8 A glance at the inner workings of an LSTM node. Information is forward-propagated through the node from input $\boldsymbol{a}_t$ to output $\boldsymbol{h}_t$ as well as passed to a subsequent instance of the node in time (from $\boldsymbol{h}_{t-1}$ and $\boldsymbol{c}_{t-1}$ to $\boldsymbol{h}_t$ and $\boldsymbol{c}_t$ respectively)

current input $\boldsymbol{a}_t$ (that equals $\boldsymbol{x}_t$ if the node is in the first hidden layer that acts upon the model's inputs):

$$h_t = f_h(h_{t-1}, \boldsymbol{a}_t) \tag{10}$$

RNNs allow for sequences in the output, input, or both, whereas MLPs are only capable of handling fixed-length vectors as input and output. For this paper, however, we chose to only output a one-step time sequence with one variable, i.e. a 1x1 matrix.

RNNs, thus, belong to the $\hat{\boldsymbol{Y}} = f(\boldsymbol{X})$ class of models, in which $\boldsymbol{Y}$ and $\boldsymbol{X}$ are matrices. The general mathematical description is similar to that of MLPs, however both inputs and outputs have an added dimension to allow for different time instances:

$$\begin{aligned} \hat{\boldsymbol{Y}} = f(\boldsymbol{X}) &= (\boldsymbol{\Psi}^m \circ \boldsymbol{\Psi}^{m-1} \circ \boldsymbol{\Psi}^{m-2} \circ \cdots \circ \boldsymbol{\Psi}^1)(\boldsymbol{X}) \\ &\text{minimizing the cost function: } \arg\min_{\boldsymbol{\Theta}} J(|\hat{\boldsymbol{Y}} - \boldsymbol{Y}|, \boldsymbol{\Theta}) \end{aligned} \tag{11}$$

RNNs can be difficult to train: the backpropagation algorithm must be adapted to backpropagate through time as well, which can lead to vanishing or exploding gradients problems. We will not elaborate on those here. By making a more sophisticated function f_h for the hidden state, Long Short-Term Memory models (LSTMs) [13]

were created. They suffer less from the aforementioned problems and are capable of learning long-term dependencies. Data scientists use LSTMs more than any other type of RNNs. Figure 8 provides a schematic overview of an LSTM node's inner workings. The formulas for an LSTM memory cell demonstrate their high complexity compared to MLP cells.[7] Fortunately, these formulas are usually integrated in the deep learning software packages and need not concern users:

$$\begin{aligned}
\boldsymbol{i}_t &= \sigma(\boldsymbol{W}_i \boldsymbol{h}_{t-1} + \boldsymbol{V}_i \boldsymbol{a}_t + \boldsymbol{b}_i)\text{: input gate} \\
\boldsymbol{f}_t &= \sigma(\boldsymbol{W}_f \boldsymbol{h}_{t-1} + \boldsymbol{V}_f \boldsymbol{a}_t + \boldsymbol{b}_f)\text{: forget gate} \\
\boldsymbol{o}_t &= \sigma(\boldsymbol{W}_o \boldsymbol{h}_{t-1} + \boldsymbol{V}_0 \boldsymbol{a}_t + \boldsymbol{b}_o)\text{: output gate} \\
\boldsymbol{g}_t &= \tanh(\boldsymbol{W}_g \boldsymbol{h}_{t-1} + \boldsymbol{V}_g \boldsymbol{a}_t + \boldsymbol{b}_g)\text{: internal hidden state} \\
\boldsymbol{c}_t &= (\boldsymbol{c}_{t-1} \otimes \boldsymbol{f}_t) \oplus (\boldsymbol{g}_t \otimes \boldsymbol{i}_t)\text{: memory cell} \\
\boldsymbol{h}_t &= \tanh(\boldsymbol{c}_t \otimes \boldsymbol{o}_t)\text{: hidden state}
\end{aligned} \tag{12}$$

References

1. Copeland, T., Koller, T., & Murrin, J. (1996). *Valuation: Measuring and managing the value of companies* (p. 550). New York: Wiley.
2. Arnold, A., Clubb, C., Manson, S., & Wearing, R. (2012). The relationship between earnings, funds flows and cash flows: Evidence for the UK. *Accounting and Business Research*, *22*(1), 13–19.
3. Akinyomi, O. (2014). Effect of cash management on profitability of Nigerian manufacturing firms. *International Journal of Marketing and Technology*, *4*(1), 129–140.
4. Garcia-Teruel, P. J. (2005). Effects of working capital management on SME profitability. *International Journal of Managerial Finance*, *3*(2), 164–177.
5. Cheng, M.-Y., Hoang, N.-D., & Wu, Y.-W. (2012). *Prediction of project cash flow using time-dependent evolutionary LS-SVM inference model*. https://pdfs.semanticscholar.org/c16b/56e6128f8c69880356ed15d6ad2f9434aa04.pdf. Last visited 2019-07-01.
6. Hu, W.-K. (2016). *Overdue invoice forecasting and data mining*. https://dspace.mit.edu/bitstream/handle/1721.1/104327/958280271-MIT.pdf?sequence=1&isAllowed=y. Last visited 2019-07-01.
7. Hu, P.-G. (2015). *Predicting and improving invoice-to-cash collection through machine learning*. https://dspace.mit.edu/bitstream/handle/1721.1/99584/925473704-MIT.pdf?sequence=1. Last visited 2019-07-01.
8. Reddy, V. (2018). *Data analysis course, time series analysis and forecasting*. http://www.trendwiseanalytics.com/training/Timeseries_Forecasting.pdf. Last visited 2019-07-01.
9. Nau, R. (2018). *Introduction to ARIMA: Nonseasonal models*. https://people.duke.edu/~rnau/411arim.htm. Last visited 2019-07-01.
10. Taylor, S. J., & Letham, B. (2017). *Prophet: Forecasting at scale*. https://research.fb.com/prophet-forecasting-at-scale/. Last visited 2019-07-01.
11. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. Cambridge: MIT Press.
12. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*, 533–536.
13. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9–8*, 1735–1780.
14. Brownlee, J. (2017). *Long short-term memory networks with python*. https://machinelearningmastery.com/lstms-with-python/

[7] Since the formula for each cell in a layer is identical, we used a vector notation to encompass all cells. $\otimes$ and $\oplus$ denote an element-wise product (Hadamard) and sum, respectively.

15. Brownlee, J. (2017). *On the suitability of long short-term memory networks for time series forecasting*. https://machinelearningmastery.com/suitability-long-short-term-memory-networks-time-series-forecasting/. Last visited 2019-07-01.
16. Upadhyay, R. (2018). *Step-by-step graphic guide to forecasting through ARIMA modeling using R—Manufacturing case study example (part 4)*. http://ucanalytics.com/blogs/step-by-step-graphic-guide-to-forecasting-through-ARIMA-modeling-in-r-manufacturing-case-study-example/. Last visited 2019-07-01.
17. Choudhary, A. (2018). *Generate quick and accurate time series forecasts using Facebook's prophet (with python and R codes)*. https://www.analyticsvidhya.com/blog/2018/05/generate-accurate-forecasts-facebook-prophet-python-r/. Last visited 2019-07-01.
18. Goyal, A., Krishnamurthy, S., Kulkarni, S., Kumar, R., Vartak, M., & Lanham, M. A. (2016). Solution to forecast demand using long short-term memory recurrent neural networks for time series forecasting. https://mwdsi2018.exordo.com/files/papers/70/final_draft/LSTM_Final_Paper_MWDSI.pdf. Last visited 2019-07-01.
19. Cheng, Y., Xu, C., Mashima, D., & Wu, Y. (2017). *PowerLSTM: Power demand forecasting using long short-term memory neural networks, advanced data mining and applications: 13th International conference, ADMA 2017, Singapore, proceedings*.
20. Neil, D., Pfeiffer, M., & Liu, S.-C. (2016). Phased LSTM: Accelerating recurrent network training for long or event-based sequences. In *30th conference on neural information processing systems (NIPS 2016)*, Barcelona, Spain.
21. Kingma, D. P., & Ba, J. L. (2015). Adam: A method for stochastic optimization. In *3rd international conference for learning representations*, San Diego.
22. Sarle, W. S. (2000). *How to measure importance of inputs?*ftp://ftp.sas.com/pub/neural/importance.html. *Last visited 2019-07-01*.
23. Schwartz-Ziv, R., & Tishby, N. (2017). *Opening the black box of deep neural networks via information, Why and when deep learning works: Looking inside deep learning ICRI-CI paperbundle*.
24. Molnar C. (2019). *Interpretable machine learning: A guide for making black box models explainable*. https://christophm.github.io/interpretable-ml-book/. Last visited 2019-07-01.
25. Fama, E. F., & Miller, M. H. (1972). *The theory of finance*. New York: The Dryden Press.
26. Barrons, J. T. (2018). *A more general robust loss function*. https://arxiv.org/pdf/1701.03077.pdf. Last visited 2019-07-01.
27. Wang, Q., Yu, J., & Deng, W. (2019). An adjustable re-ranking approach for improving the individual and aggregate diversities of product recommendations. *Electronic Commerce Research*, *19*, 1. https://doi.org/10.1007/s10660-018-09325-4.
28. Liu, S., Shao, B., Gao, Y., et al. (2018). Game theoretic approach of a novel decision policy for customers based on big data. *Electronic Commerce Research*, *18*(2), 2017. https://doi.org/10.1007/s10660-017-9259-6.
29. Gong, K., Peng, Y., Wang, Y., et al. (2018). Time series analysis for C2C conversion rate. *Electronic Commerce Research*, *18*, 4. https://doi.org/10.1007/s10660-017-9283-6. last visited 2019-07-01 (2017).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.